Task1

- **What do you notice in the ordering of the output messages? Under which assumptions is non-blocking communication better than blocking communication?**

Case 1 — 1 node, 1 task-per-node → 1 rank

- Only one process exists
- No communication partners
- No message exchange possible
  ➡️ **Not useful for MPI message exchange**

Case 2 — 1 node, 2 task-per-node → 2 ranks, both on same node

- Two MPI processes
- But both run on the **same machine**
- Communication happens inside a single host memory domain
  ➡️ **Still not demonstrating multi-node communication**
  (MPI is *distributed-memory* — needs multiple nodes)

Case 2 — 1 node, 2 task-per-node → 2 ranks, both on same node

- Two MPI processes
- But both run on the **same machine**
- Communication happens inside a single host memory domain
  ➡️ **Still not demonstrating multi-node communication**
  (MPI is *distributed-memory* — needs multiple nodes)

Case 3 — 2 nodes, 1 task-per-node → 2 ranks on 2 separate nodes

This is the **correct** configuration for a real MPI scenario:

- Rank 0 on node A
- Rank 1 on node B
- Communication goes through the cluster interconnect (InfiniBand)
  ➡️ **This configuration makes sense for MPI communication**

Case 4 — 2 nodes, 2 tasks-per-node → 4 ranks in total

- 2 ranks on node A

- 2 ranks on node B
  This *does* use multiple nodes, but:
- Ranks on the same node communicate through shared memory
- Only 2 ranks communicate across nodes
  ➡️ **Valid, but unnecessarily complex for beginners**

**Only the configurations that use ≥2 nodes are meaningful for demonstrating MPI-based inter-node message exchange. This means:

- "2 nodes, 1 task-per-node" is the ideal configuration
- "2 nodes, 2 tasks-per-node" works but mixes shared-memory and distributed-memory communication
- Both single-node configurations do NOT demonstrate real MPI message passing between machines.**

Task2

- **What happens if you increase ntasks-per-node? Explain the resulting behavior**

When ntasks-per-node is increased, more MPI processes run on the same node. This causes a mixture of intra-node and inter-node communication, which is not the goal of the exercise. It also leads to more chaotic output, more scheduling overhead, reduced performance, and does not demonstrate true distributed MPI behavior.

T**ask3**

- **What do you notice in the ordering of the output messages? Under which assumptions is non-blocking communication better than blocking communication?**

Non-blocking MPI results in nondeterministic output ordering because messages may arrive at different times and processes do not wait at send/receive time.
Non-blocking communication is better when we want to overlap computation with communication, avoid deadlocks, hide network latency, and achieve higher throughput by allowing multiple concurrent message transfers.

Task4

- **How does this approach compare to blocking and non-blocking communication?**

What are the advantages and disadvantages?

MPI_Bcast is a collective communication operation that broadcasts a message from the root rank to all ranks at once. Compared to blocking point-to-point communication, it is much more efficient, prevents deadlocks, and requires far less code because the root does not need to send separate messages to every process. Compared to non-blocking communication, MPI_Bcast is simpler and guarantees that all ranks receive the data at the same point in execution, but it does not allow overlapping computation with communication. Its main advantages are scalability, simplicity, deadlock avoidance, and predictable synchronization. Its disadvantages are that all ranks must participate, it cannot express more flexible communication patterns, and it does not allow fine-grained overlap of communication and computation